

Operatorii ROLLUP și CUBE. Clauza GROUPING SETS. Funcția GROUPING. Cereri ierarhice. Clauza WITH.

I. [Operatorii ROLLUP și CUBE. Clauza GROUPING SETS. Funcția GROUPING.]

• **[Operatorul ROLLUP]** Acest operator furnizează valori **agregat** și **superagregat** corespunzătoare expresiilor din clauza *GROUP BY*. Operatorul *ROLLUP* poate fi folosit pentru extragerea de statistici și informații totalizatoare din mulțimile rezultate. Acest operator poate fi util la generarea de rapoarte, diagrame și grafice.

Operatorul *ROLLUP* creează grupări prin deplasarea într-o singură direcție, de la dreapta la stânga, de-a lungul listei de coloane specificate în clauza *GROUP BY*. Apoi, se aplică funcția agregat acestor grupări. Dacă sunt specificate ***n* expresii** în operatorul *ROLLUP*, numărul de grupări generate va fi ***n* + 1**. Liniile care se bazează pe valoarea primelor *n* expresii se numesc linii obișnuite, iar celelalte se numesc linii superagregat.

Dacă în clauza *GROUP BY* sunt specificate *n* coloane, pentru a produce subtotaluri fără operatorul *ROLLUP* ar fi necesare *n* + 1 instrucțiuni *SELECT* conectate prin *UNION ALL*. Aceasta ar face execuția cererii ineficientă pentru că fiecare instrucțiune *SELECT* determină accesarea tabelului. Operatorul *ROLLUP* determină rezultatele efectuând un singur acces la tabel și este util atunci când sunt implicate multe coloane în producerea subtotalurilor.

Ilustrăm aplicarea acestui operator prin urmatorul exemplu.

Exemplu:

Pentru departamentele având codul mai mic decât 50, să se afișeze:

- pentru fiecare departament și pentru fiecare an al angajării (corespunzător departamentului respectiv), valoarea totală a salariilor angajaților în acel an;
- valoarea totală a salariilor pe departamente (indiferent de anul angajării);
- valoarea totală a salariilor (indiferent de anul angajării și de departament).

```
SELECT department_id, TO_CHAR(hire_date, 'yyyy'), SUM(salary)
FROM employees
WHERE department_id < 50
GROUP BY ROLLUP(department_id, TO_CHAR(hire_date, 'yyyy'));
```

Instrucțiunea precedentă va avea un rezultat de forma:

DEPARTMENT_ID	TO_CHAR(hire_date,'yyyy')	SUM(SALARY)
10	1987	4400
10		4400
20	1996	13000
20	1997	6000
20		19000
30	1994	11000
30	1995	3100
30	1997	5700
30	1998	2600
30	1999	2500
30		24900
40	1994	6500
40		6500
		54800

În rezultatul prezentat anterior se pot distinge 3 tipuri de linii.

- Prima linie reprezintă suma salariilor angajaților în 1987 din departamentul care are codul 10. În mod similar se interpretează liniile din rezultat care au toate coloanele completate.
- Linia a doua conține valoarea totală a salariilor din departamentul al cărui cod este 10. La fel se interpretează toate liniile care se disting prin faptul că valoarea coloanei *TO_CHAR(hire_date, 'dd')* este *null*.
- Ultima linie conține suma salariilor tuturor angajaților din departamentele al căror cod este mai mic decât 50. Întrucât această linie corespunde totalului general, ea conține valoarea *null* pe toate coloanele, cu excepția câmpului *SUM(salary)*.

• [Operatorul CUBE]

Operatorul *CUBE* grupează liniile selectate pe baza valorilor tuturor combinațiilor posibile ale expresiilor specificate și returnează câte o linie totalizatoare pentru fiecare grup. Acest operator este folosit pentru a produce mulțimi de rezultate care sunt utilizate în rapoarte. În vreme ce *ROLLUP* produce subtotalurile doar pentru o parte dintre combinațiile posibile, operatorul *CUBE* produce subtotaluri pentru **toate combinațiile posibile** de grupări specificate în clauza *GROUP BY*, precum și un total general.

Dacă există ***n* coloane** sau expresii în clauza *GROUP BY*, vor exista ***2ⁿ* combinații** posibile superagregat. Din punct de vedere matematic, aceste combinații formează un cub *n*-dimensional, de aici provenind numele operatorului. Pentru producerea de subtotaluri fără ajutorul operatorului *CUBE* ar fi necesare *2ⁿ* instrucțiuni *SELECT* conectate prin *UNION ALL*.

Exemplu:

Pentru departamentele având codul mai mic decât 50 să se afișeze:

- valoarea totală a salariilor corespunzătoare fiecărui an de angajare, din cadrul fiecărui departament;
- valoarea totală a salariilor din fiecare departament (indiferent de anul angajării);
- valoarea totală a salariilor corespunzătoare fiecărui an de angajare (indiferent de departament);
- valoarea totală a salariilor (indiferent de departament și de anul angajării).

```
SELECT department_id, TO_CHAR(hire_date, 'yyyy'), SUM(salary)
FROM employees
WHERE department_id < 50
GROUP BY CUBE(department_id, TO_CHAR(hire_date, 'yyyy'));
```

În plus față de rezultatul corespunzător operației *ROLLUP*, operatorul *CUBE* va produce linii care reprezintă suma salariilor pentru fiecare an de angajare corespunzător unui departament având codul mai mic decât 50. Aceste linii se disting prin faptul că valoarea coloanei *department_id* este *null*.

- Pentru determinarea modului în care a fost obținută o valoare totalizatoare cu *ROLLUP* sau *CUBE*, se utilizează funcția:
- **GROUPING**(expresie)
Aceasta întoarce:
 - valoarea 0, dacă expresia a fost utilizată pentru calculul valorii agregat
 - valoarea 1, dacă expresia nu a fost utilizată.
- Dacă se dorește obținerea numai a anumitor grupări superagregat, acestea pot fi precizate prin intermediul clauzei :
 - **GROUPING SETS** ((expr_11, expr_12, ..., expr_1n), (expr_21, expr_22, ...expr_2m), ...)

- Operatorul **EXISTS**
- În instrucțiunile *SELECT* imbricate, este permisă utilizarea oricărui operator logic.
- Pentru a testa dacă valoarea recuperată de cererea externă există în mulțimea valorilor regăsite de cererea internă corelată, se poate utiliza operatorul *EXISTS*. Dacă subcererea returnează cel puțin o linie, operatorul returnează valoarea *TRUE*. În caz contrar, va fi returnată valoarea *FALSE*.
- Operatorul *EXISTS* asigură că nu mai este continuată căutarea în cererea internă după ce aceasta regăsește o linie.

II. [Subcereri ierarhice]

- Clauzele **START WITH** și **CONNECT BY** se utilizează în formularea cererilor ierarhice.
 - *START WITH* specifică o condiție care identifică liniile ce urmează să fie considerate ca rădăcini ale cererii ierarhice respective. Dacă se omite această clauză, sistemul *Oracle* utilizează toate liniile din tabel drept linii rădăcină.
 - *CONNECT BY* specifică o condiție care identifică relația dintre liniile „părinte” și „copil” ale ierarhiei. Condiția trebuie să conțină operatorul *PRIOR* pentru a face referință la linia „părinte”.
 - Operatorul *PRIOR* face referință la linia „părinte”. Plasarea acestui operator determină direcția interogării, dinspre „părinte” spre „copil” (*top-down*) sau invers (*bottom-up*). Traversarea *top-down*, respectiv *bottom-up* a arborelui se realizează prin specificări de forma următoare:

Top-down: CONNECT BY PRIOR cheie_parinte = cheie_copil;

Bottom-up: CONNECT BY PRIOR cheie_copil = cheie_parinte;

Obs: Operatorul *PRIOR* poate fi plasat în fața oricărui membru al condiției specificate în clauza *CONNECT BY*.

Obs: Liniile „părinte” ale interogării sunt identificate prin clauza *START WITH*. Pentru a găsi liniile „copil”, server-ul evaluează expresia din dreptul operatorului *PRIOR* pentru linia „părinte”, și cealaltă expresie pentru fiecare linie a tabelului. Înregistrările pentru care condiția este adevărată vor fi liniile „copil”. Spre deosebire de *START WITH*, în clauza *CONNECT BY* nu pot fi utilizate subcereri.

- Pseudocoloana **LEVEL** poate fi utilă într-o cerere ierarhică. Aceasta determină lungimea drumului de la rădăcină la un nod.

III. [Clauza WITH]

- Cu ajutorul clauzei *WITH* se poate defini un bloc de cerere înainte ca acesta să fie utilizat într-o interogare.
- Clauza permite reutilizarea aceluiași bloc de cerere într-o instrucțiune *SELECT* complexă. Acest lucru este util atunci când o cerere face referință de mai multe ori la același bloc de cerere, care conține operații *join* și funcții agregat.

Exerciții:

- a) Să se afișeze numele departamentelor, titlurile job-urilor și valoarea medie a salariilor, pentru:
 - fiecare departament și, în cadrul său pentru fiecare job;
 - fiecare departament (indiferent de job);
 - întreg tabelul.

b) Analog cu a), afișând și o coloană care arată intervenția coloanelor *department_name*, *job_title*, în obținerea rezultatului.

2. a) Să se afișeze numele departamentelor, titlurile job-urilor și valoarea medie a salariilor, pentru:
 - fiecare departament și, în cadrul său pentru fiecare job;
 - fiecare departament (indiferent de job);
 - fiecare job (indiferent de departament)
 - întreg tabelul.
- b) Cum intervin coloanele în obținerea rezultatului? Să se afișeze 'Dep', dacă departamentul a intervenit în agregare, și 'Job', dacă job-ul a intervenit în agregare.
3. Să se afișeze numele departamentelor, numele job-urilor, codurile managerilor, maximul și suma salariilor pentru:
 - fiecare departament și, în cadrul său, fiecare job;
 - fiecare job și, în cadrul său, pentru fiecare manager;
 - întreg tabelul.
4. Să se afișeze salariul maxim al angajaților doar dacă acesta este mai mare decât 15000.
5. Să se obțină numele salariaților care lucrează într-un departament în care există cel puțin un angajat cu salariul egal cu salariul maxim din departamentul 30.
6. Să se afișeze codul, numele și prenumele angajaților care au cel puțin doi subalterni.
7. Să se determine locațiile în care se află cel puțin un departament (**care este cea mai simplă soluție?**).
8. Să se afișeze codul, numele, data angajării, salariul și managerul pentru:
 - a) **subalternii direcți ai lui De Haan**;
 - b) **ierarhia arborescentă de sub De Haan**.
9. Să se obțină ierarhia șef-subaltern, considerând ca rădăcină angajatul având codul 114.
10. Scrieți o cerere ierarhică pentru a afișa codul salariatului, codul managerului și numele salariatului, pentru angajații care sunt cu 2 niveluri sub De Haan. Afișați, de asemenea, nivelul angajatului în ierarhie.
11. Pentru fiecare linie din tabelul EMPLOYEES se va afișa o structură arborescentă în care va apărea angajatul, managerul său, managerul managerului etc. Coloanele afișate vor fi: codul angajatului, codul managerului, nivelul în ierarhie (LEVEL) și numele angajatului. Se vor folosi indentări (**cine este angajatul care apare de cele mai multe ori în rezultat? de câte ori apare?**)
12. Pentru fiecare angajat să se determine numărul de subalterni (**nu doar cei direcți**).
13. Utilizând clauza *WITH*, să se scrie o cerere care afișează numele departamentelor și valoarea totală a salariilor din cadrul acestora. Se vor considera departamentele a căror valoare totală a salariilor este mai mare decât media valorilor totale ale salariilor pe departamente.
14. Să se afișeze ierarhic codul, prenumele și numele (pe aceeași coloană), codul job-ului și data angajării, pornind de la **subordonații direcți ai lui Steven King** care au cea mai mare vechime.
15. Să se afișeze informații despre departamente, în formatul următor: „Departamentul <department_name> este condus de {<manager_id> | nimeni} și {are numărul de salariați <n> | nu are salariați}”.
16. Să se afișeze:
 - suma salariilor, pentru job-urile care încep cu litera S;
 - media generală a salariilor, pentru job-ul având salariul maxim;
 - salariul minim, pentru fiecare din celelalte job-uri.